

Wrapping AWS Lambda ESM handler

The problem: How to transform AWS Lambda handler below?

```
// input.js
export const handler = async(event) => {
  return "Hi from AWS Lambda";
};
```

To the following, notice the WrapAwsLambda wrapper:

```
// transformed.js
export const handler = WrapAwsLambda(async(event) => {
  return "Hi from AWS Lambda";
});
```

Using loader hooks

Built-in Node.js mechanism to hook into the ESM `import` :

<https://nodejs.org/api/module.html#customization-hooks>

Asynchronous hooks

```
// register_async_hooks.mjs
import { register } from "node:module";
register("./async-hooks.mjs", import.meta.url);
```

Add `--import` to register hooks:

```
node --import ./register_async_hooks.mjs runtime.mjs
```

The hooks `load` function runs on a dedicated thread!

Async hooks **load** function

```
// async_hooks.mjs
import { transformLambda } from "../myTransform.js";
let patched = false;
export async function load(url, context, nextLoad) {
  const result = await nextLoad(url, context);
  if (!patched && url.endsWith("/handler.mjs")) {
    patched = true;
    return {
      format: "module", shortCircuit: true,
      source: transformLambda(result.source.toString(), "handler", "WrapAwsLambda")
    };
  }
  return result;
}
```

Note: async hooks are usually 2x slower than sync hooks.

Synchronous hooks **load** function

```
// sync_hooks.mjs
import { registerHooks } from "node:module";
import { transformLambda } from "../myTransform.js";
let patched = false;
registerHooks({
  load(url, context, nextLoad) {
    const result = nextLoad(url, context);
    if (!patched && url.endsWith("/handler.mjs")) {
      patched = true;
      return {
        format: "module",
        shortCircuit: true,
        source: transformLambda(result.source.toString(), "handler", "WrapAwsLambda")
      };
    }
    return result;
  },
});
```

Transforming ESM source code

Naive approach using `RegExp()` for direct string manipulation:

```
// regex-transform.ts
export function transformLambda(input: string, handler: string, wrapper: string): string {
  return input.replace(
    new RegExp(`export const ${handler} = (.+);`, "s"),
    `export const ${handler} = ${wrapper}($1);`
  );
}
```

This is the fastest way, which we will use as baseline for AST transformations.

Babel transform

Babel transformation with custom `visitor` is used to generate the wrapper call:

```
// babel-transform.ts
ExportNamedDeclaration(path) {
  if (t.isVariableDeclaration(path.node.declaration)) {
    const varDecl = path.node.declaration.declarations[0];
    if (t.isIdentifier(varDecl.id) && varDecl.id.name === handler) {
      path.replaceWith(t.exportNamedDeclaration(
        t.variableDeclaration("const", [t.variableDeclarator(t.identifier(handler),
          t.callExpression(t.identifier(wrapper), [varDecl.init!]))]),
      ));
      path.skip();
    }
  }
}
```

Acorn => estraverse => astring

Acorn is only a parser, `estraverse` and `astring` adds traversing and codegen.

```
const ast = acorn.parse(code, { ecmaVersion: 2020, sourceType: "module" });
estraverse.replace(ast as ESTree.Node, {
  enter: function (node) {
    if (node.type === "ExportNamedDeclaration"
        && node.declaration?.type === "VariableDeclaration") {
      const varDecl = node.declaration.declarations[0];
      if (varDecl.id.type === "Identifier" && varDecl.id.name === handler) {
        varDecl.init = NESTree.CallExpression(Helpers.AutoChain(wrapper),
            [varDecl.init as NESTree.Expression]) as ESTree.Expression;
      }
      return { ...node };
    }
  }
});
return astring.generate(ast);
```

Rust native addon using `oxc.rs`

The `oxc.rs` tools and crate `oxc_transformer` provides API similar to Babel in Rust.

With `napi.rs` we can easily export the Rust code:

```
use napi_derive::napi;
mod transform;

#[napi]
pub fn transform_lambda(input: String) -> String {
    transform::transform_lambda_source(input)
}
```

```
const output = transformLambda(input);
```


Rust wasm plugin for `swc.rs`

Based on the [SWC](#) guide for [plugin creation](#) compiled to WebAssembly.

```
// swc-wrapper.cjs
const swc = require("@swc/core");
exports.transformLambda = function (sourceCode, handler, wrapper) {
  const output = swc.transformSync(sourceCode, {
    filename: "handler.mjs", sourceMaps: false, isModule: true, {
      target: "esnext", experimental: {
        plugins: [[ require.resolve("./my-plugin.wasm"), { handler, wrapper } ]]
      }
    },
  });
  return output.code;
};
```

Benchmark

Benchmark table via [hyperfine](#) and sync hooks:

Hooks	Mean [ms]	Min [ms]	Max [ms]	Relative
N/A	20.9 ± 1.9	16.8	26.2	1.00
RegExp	24.9 ± 3.8	18.0	36.0	1.19 ± 0.21
oxc.rs	33.6 ± 2.6	27.9	39.2	1.61 ± 0.19
Acorn	47.0 ± 5.2	39.1	63.4	2.25 ± 0.32
swc.rs	122.1 ± 6.9	110.9	143.1	5.85 ± 0.62
Babel	142.2 ± 4.1	134.0	148.2	6.81 ± 0.64